PYTHON DATA ANALYSIS INTERVIEW QUESTIONS & ANSWERS

A practical preparation guide for fresher and entry-level Data Analyst roles

NumPy • Pandas • Data Cleaning • EDA • Visualization • Statistics • Python Coding • Scenario Questions

How to use this guide

For each question, learn the concept, explain why it matters, and practise the code without copying. Follow-up questions test whether you understand the limitations and assumptions behind a method.

> Interview tip: describe the problem, the method you would choose, why it is appropriate, and how you would validate the result.

Examples use common Python libraries: NumPy, Pandas, Matplotlib and Seaborn. Install them in your Python environment before running the examples.

Contents

1. NumPy: core questions and follow-ups
2. Pandas: core questions and follow-ups
3. EDA, visualization and chart selection
4. Core Python questions
5. Data cleaning and practical Pandas questions
6. Statistics and analytical reasoning
7. Python coding questions for analysts
8. Scenario-based interview questions
9. Rapid revision: short questions
10. Final preparation checklist

Note: interview questions vary by company and role. This guide focuses on common, high-value topics for entry-level Python Data Analyst interviews; it is not a guarantee of every question any interviewer may ask.

1. NumPy: core questions and follow-ups

1. What is NumPy, and why is it used in Python?

Answer: NumPy (Numerical Python) provides efficient multidimensional arrays and numerical operations. Analysts use it for vectorized calculations, statistics, mathematical transformations, and as a foundation for many data-science libraries.

```
import numpy as np

sales = np.array([100, 200, 300, 400])
print(np.mean(sales))  # 250.0
print(np.sum(sales))   # 1000
```

Follow-up / counter-question: Why can NumPy be faster than a Python list for numerical work?

Suggested answer: Many NumPy operations run in optimized compiled code and avoid Python-level loops. Speed depends on data size, operation, and type.

Follow-up / counter-question: Is NumPy only used in data analysis?

Suggested answer: No. It is also used in scientific computing, simulations, engineering, image processing, and machine learning.

Follow-up / counter-question: Is a NumPy array always faster than a list?

Suggested answer: No. Small or irregular operations may not benefit; compare performance for the actual workload.

2. What is the difference between a Python list and a NumPy array?

Answer: A Python list is a general-purpose container that can hold mixed object types. A NumPy array is designed for efficient numerical operations and normally has one common dtype. Array arithmetic is vectorized, while list arithmetic often needs a loop or comprehension.

```
values = [10, 20, 30]
print(values * 2)  # repeats the list

arr = np.array([10, 20, 30])
print(arr * 2)     # [20 40 60]
```

Follow-up / counter-question: What happens when you multiply a list by 2?

Suggested answer: It repeats the list; it does not multiply each element.

Follow-up / counter-question: Can a NumPy array hold mixed strings and numbers?

Suggested answer: Yes, but NumPy may coerce them to a common dtype, such as strings.

Follow-up / counter-question: When would you use a list instead?

Suggested answer: For general collections, irregular nested structures, or objects that do not need numerical vectorization.

3. What is the difference between 1D, 2D and 3D arrays?

Answer: A 1D array has one axis, a 2D array has rows and columns, and a 3D array has three axes, often interpreted as a stack of 2D arrays. The number of dimensions is returned by `ndim`; the length along each axis is returned by `shape`.

```
a = np.array([1, 2, 3, 4])
b = np.array([[1, 2, 3], [4, 5, 6]])
c = np.zeros((2, 3, 4))
print(a.shape, a.ndim)  # (4,), 1
print(b.shape, b.ndim)  # (2, 3), 2
print(c.shape, c.ndim)  # (2, 3, 4), 3
```

Follow-up / counter-question: What is the shape of a 1D array with four values?

Suggested answer: `(4,)`, not `(4, 1)`. The latter is a two-dimensional column array.

Follow-up / counter-question: How many dimensions does an array with shape (2, 3, 4) have?

Suggested answer: Three dimensions, with 24 elements in total.

4. How do you create a NumPy array?

Answer: Use `np.array()` to create an array from existing values. Other common functions include `zeros`, `ones`, `full`, `arange`, `linspace` and `eye`.

```
np.array([1, 2, 3])
np.zeros((2, 3))
np.ones((2, 2))
np.full((2, 2), 7)
np.arange(1, 10, 2)
np.linspace(0, 1, 5)
np.eye(3)
```

Follow-up / counter-question: What is the difference between `zeros()` and `empty()`?

Suggested answer: `zeros()` initializes values to zero; `empty()` does not initialize them, so its contents should not be assumed to be zero.

Follow-up / counter-question: How do you create a 3 by 3 identity matrix?

Suggested answer: Use `np.eye(3)`.

5. What is the difference between `np.array()` and `np.arange()`?

Answer: `np.array()` builds an array from values you supply. `np.arange(start, stop, step)` generates a range of values; the stop value is excluded.

```
print(np.array([2, 4, 6, 8]))
print(np.arange(2, 10, 2))
print(np.linspace(0, 10, 5))
```

Follow-up / counter-question: How is `linspace()` different?

Suggested answer: `linspace(start, stop, num)` creates a specified number of evenly spaced values and includes the endpoint by default.

Follow-up / counter-question: Why be careful with floating-point `arange()`?

Suggested answer: Floating-point precision can produce unexpected endpoints or counts. `linspace()` is often better when the desired number of points is known.

6. What is broadcasting in NumPy?

Answer: Broadcasting lets NumPy perform element-wise operations on compatible shapes without manually repeating values. Shapes are compared from the right; each pair of dimensions must be equal or one of them must be 1. Missing leading dimensions are treated as 1.

```
prices = np.array([100, 200, 300])
print(prices - 10)  # [90 190 290]

sales = np.array([[100, 200, 300],
                  [400, 500, 600]])
adjustment = np.array([10, 20, 30])
print(sales + adjustment)
```

Follow-up / counter-question: Why can broadcasting fail for shapes (2, 3) and (2,)?

Suggested answer: The trailing dimensions 3 and 2 conflict. Reshape the second array if the intended calculation is row-wise or column-wise.

Follow-up / counter-question: What is a simple scalar broadcasting example?

Suggested answer: Subtracting a scalar from an array applies that scalar to every element.

7. How do you find the shape and dimensions of an array?

Answer: `shape` returns the size along each axis; `ndim` returns the number of axes; `size` returns the total number of elements; and `dtype` returns the data type.

```
arr = np.array([[10, 20, 30], [40, 50, 60]])
print(arr.shape)  # (2, 3)
print(arr.ndim)   # 2
print(arr.size)   # 6
print(arr.dtype)
```

Follow-up / counter-question: What is the difference between size and shape?

Suggested answer: `size` is one total count; `shape` gives each dimension's length.

8. How do you calculate mean, median, standard deviation and variance?

Answer: Use `np.mean`, `np.median`, `np.std` and `np.var`. Mean is the arithmetic average, median is the middle value, standard deviation measures spread in original units, and variance is the average squared deviation.

```
sales = np.array([10, 20, 30, 40, 50])
print(np.mean(sales))    # 30.0
print(np.median(sales))  # 30.0
print(np.std(sales))     # population standard deviation
print(np.var(sales))     # population variance
print(np.var(sales, ddof=1))  # sample variance
```

Follow-up / counter-question: Why can median be better than mean?

Suggested answer: Median is less sensitive to extreme values, which is useful for skewed data such as salaries.

Follow-up / counter-question: Why can NumPy and Pandas standard deviations differ?

Suggested answer: `np.std()` defaults to `ddof=0` (population form); Pandas Series `.std()` defaults to `ddof=1` (sample form).

Follow-up / counter-question: How do you calculate column-wise means in a 2D array?

Suggested answer: Use `np.mean(arr, axis=0)`; `axis=1` gives row-wise means.

9. How do you handle missing values in NumPy?

Answer: Floating-point arrays commonly represent missing numerical values with `np.nan`. Use `np.isnan()` to identify them and functions such as `np.nanmean()` or `np.nanmedian()` to calculate while ignoring NaNs. Replacing missing values requires a reasoned decision.

```
data = np.array([10.0, 20.0, np.nan, 40.0])
print(np.isnan(data))
print(np.nanmean(data))
mean_value = np.nanmean(data)
data[np.isnan(data)] = mean_value
```

Follow-up / counter-question: Does `np.mean()` ignore NaN automatically?

Suggested answer: No. It generally returns NaN if the input includes NaN; `np.nanmean()` ignores NaN values.

Follow-up / counter-question: Should missing values always be replaced by the mean?

Suggested answer: No. Investigate why values are missing and consider median, a meaningful category, an indicator, or leaving them missing.

2. Pandas: core questions and follow-ups

10. What is Pandas, and why is it used for data analysis?

Answer: Pandas provides Series and DataFrame structures for loading, inspecting, cleaning, transforming, aggregating and combining structured data. It is central to many repeatable Python analysis workflows.

```
import pandas as pd

df = pd.DataFrame({
    "city": ["Vadodara", "Ahmedabad", "Surat"],
    "sales": [10000, 15000, 12000]
})
print(df["sales"].mean())
print(df.groupby("city")["sales"].sum())
```

Follow-up / counter-question: Why use Pandas instead of Excel?

Suggested answer: Pandas supports automation, reproducible transformations, complex joins and integration with Python and SQL. Excel remains useful for manual exploration and collaboration.

Follow-up / counter-question: Can Pandas handle any dataset size?

Suggested answer: No. Standard Pandas workflows generally need data to fit in memory; large datasets may require chunking, a database or distributed tools.

11. What is the difference between a Series and a DataFrame?

Answer: A Series is a one-dimensional labeled array. A DataFrame is a two-dimensional table with row and column labels.

```
series = df["sales"]
one_col_df = df[["sales"]]
print(type(series))
print(type(one_col_df))
```

Follow-up / counter-question: What is the difference between `df['sales']` and `df[['sales']]`?

Suggested answer: The first returns a Series; the second returns a one-column DataFrame.

12. How do you read a CSV file using Pandas?

Answer: Use `pd.read_csv('file.csv')`. Parameters such as `usecols`, `encoding`, `na_values`, `dtype` and `chunksize` help control loading.

```
df = pd.read_csv("sales.csv")
selected = pd.read_csv("sales.csv", usecols=["order_id", "sales"])

for chunk in pd.read_csv("large.csv", chunksize=10000):
    print(chunk.shape)
```

Follow-up / counter-question: How do you load only selected columns?

Suggested answer: Pass a list of names to `usecols`.

Follow-up / counter-question: What if the file is too large to load at once?

Suggested answer: Use `chunksize` and process each chunk, aggregating results as needed.

13. How do you check the first and last few rows?

Answer: `head()` shows the first rows, `tail()` shows the last rows, and `sample()` returns a random sample. Use more than `head()` to inspect data because the first rows may not be representative.

```
df.head()
df.head(10)
df.tail()
df.sample(5, random_state=42)
```

Follow-up / counter-question: Why use `sample()` during cleaning?

Suggested answer: Random samples may reveal formats or anomalies not visible at the top of a dataset.

14. How do you check data types and basic information?

Answer: Use `info()`, `dtypes`, `shape`, `describe()` and `describe(include='all')`. They help identify types, non-null counts, dimensions, distributions and category summaries.

```
df.info()
print(df.dtypes)
print(df.shape)
print(df.describe(include="all"))

df["order_date"] = pd.to_datetime(
    df["order_date"], errors="coerce"
)
```

Follow-up / counter-question: What if a date column is stored as text?

Suggested answer: Use `pd.to_datetime()` after checking the source format; `errors='coerce'` converts unparseable values to `NaT` for investigation.

Follow-up / counter-question: Why might a numeric column have object dtype?

Suggested answer: It may contain currency symbols, commas, text, mixed values or inconsistent formatting.

15. How do you identify missing values?

Answer: Use `isna()` or `isnull()` to identify missing values, `sum()` to count them, and `mean()` to calculate their proportions.

```
df.isna().sum()
df.isna().sum().sum()
missing_pct = df.isna().mean() * 100
df["city"] = df["city"].replace(r"^\s*$", pd.NA, regex=True)
```

Follow-up / counter-question: Are `isna()` and `isnull()` different?

Suggested answer: They are equivalent aliases in Pandas.

Follow-up / counter-question: Does an empty string count as missing?

Suggested answer: Usually not. Standardize blank or whitespace-only strings explicitly when they should represent missing values.

16. How do you handle missing values with `fillna()` and `dropna()`?

Answer: `fillna()` replaces missing values; `dropna()` removes rows or columns according to a missingness rule. Choose the method based on why values are missing and how the analysis will use them.

```
df["age"] = df["age"].fillna(df["age"].median())
df_clean = df.dropna(subset=["customer_id"])
df = df.fillna({"city": "Unknown"})
```

Follow-up / counter-question: When might median imputation be preferable to mean imputation?

Suggested answer: For skewed numeric distributions or data with influential outliers, though the context and missingness mechanism still matter.

Follow-up / counter-question: Why not drop every row containing a missing value?

Suggested answer: It can discard substantial data and introduce bias if missingness is systematic.

17. How do you remove duplicate records?

Answer: Use `duplicated()` to identify duplicates and `drop_duplicates()` to remove them. Choose the relevant columns based on the table's intended grain and business key.

```
df[df.duplicated(keep=False)]
df[df.duplicated(subset=["order_id"], keep=False)]
df_clean = df.drop_duplicates(subset=["order_id"], keep="first")
```

Follow-up / counter-question: What does `keep=False` mean?

Suggested answer: Every occurrence of a duplicated key is marked as duplicate.

Follow-up / counter-question: Should duplicate customer IDs always be deleted?

Suggested answer: No. An orders table may legitimately contain many rows per customer. Establish the intended grain first.

18. How do you filter rows based on a condition?

Answer: Use Boolean indexing. Use `&` for element-wise AND, `|` for OR, `~` for NOT, and parentheses around individual conditions.

```
high_sales = df[df["sales"] > 10000]
selected = df[
    (df["sales"] > 10000) & (df["status"] == "delivered")
]
cities = df[df["city"].isin(["Surat", "Vadodara"])]
```

Follow-up / counter-question: Why not use Python `and` on two Series conditions?

Suggested answer: `and` expects one truth value; a Series has many values. Use `&` for element-wise conditions.

Follow-up / counter-question: How do you filter values from a list?

Suggested answer: Use `Series.isin([...])`.

19. What is the difference between `loc[]` and `iloc[]`?

Answer: `loc[]` selects by index and column labels. `iloc[]` selects by integer positions. Label-based slices generally include the end label; positional slices exclude the end position.

```
df.loc[0, "sales"]  # label 0
df.iloc[0, 1]       # first row, second column
df.iloc[0:3, :]     # first three row positions
```

Follow-up / counter-question: What if the index labels are not 0, 1, 2, ...?

Suggested answer: `loc[0]` searches for label 0; `iloc[0]` selects the first row position.

20. How do you add, rename and delete columns?

Answer: Assign a Series or expression to create a column, use `rename(columns=...)` to rename columns, and `drop(columns=...)` to remove columns.

```
df["revenue"] = df["price"] * df["quantity"]
df = df.rename(columns={"revenue": "total_revenue"})
df = df.drop(columns=["temporary_column"])
```

Follow-up / counter-question: What is the difference between `del` and `drop()`?

Suggested answer: Both can remove columns. `del` removes directly; `drop()` returns a modified object by default unless used differently.

Follow-up / counter-question: Why prefer clear reassignment?

Suggested answer: It makes transformation steps explicit and easier to debug.

21. What is `groupby()` used for?

Answer: `groupby()` divides rows into groups and supports aggregation, transformation and filtering. It is used for summaries such as sales by region or order count by customer.

```
summary = df.groupby("city").agg(
    total_sales=("sales", "sum"),
    average_sales=("sales", "mean"),
    order_count=("order_id", "nunique")
)
df["city_avg"] = df.groupby("city")["sales"].transform("mean")
```

Follow-up / counter-question: What is the difference between aggregation and transformation?

Suggested answer: Aggregation typically reduces each group to summary rows; transformation returns group-based values aligned to the original rows.

22. What is the difference between `merge()`, `join()` and `concat()`?

Answer: `merge()` combines datasets using matching keys or columns; `join()` commonly combines on indexes; `concat()` stacks objects along rows or columns.

```
merged = orders.merge(
```

```
    customers, on="customer_id", how="left",
    validate="many_to_one", indicator=True
)
combined = pd.concat([jan_sales, feb_sales], ignore_index=True)
```

Follow-up / counter-question: Explain inner, left, right and outer joins.

Suggested answer: Inner keeps matching keys; left keeps all left keys; right keeps all right keys; outer keeps keys from either side.

Follow-up / counter-question: Why can a merge increase row count unexpectedly?

Suggested answer: Repeated keys on both sides can create multiple combinations. Check key uniqueness and use `validate=` when appropriate.

23. How do you sort by one or multiple columns?

Answer: Use `sort_values()`, setting `ascending` for the direction. Use `sort_index()` to sort index labels. Sorting rows does not normally reset their index.

```
df.sort_values("sales", ascending=False)
df.sort_values(["city", "sales"], ascending=[True, False])
df = df.sort_values("sales").reset_index(drop=True)
```

Follow-up / counter-question: How do you sort city ascending and sales descending?

Suggested answer: Use `sort_values(['city', 'sales'], ascending=[True, False])`.

Follow-up / counter-question: How do you create a fresh sequential index?

Suggested answer: Use `reset_index(drop=True)`.

3. EDA, visualization and chart selection

24. What is EDA, and why is it important?

Answer: Exploratory Data Analysis examines structure, quality, distributions and relationships before conclusions are drawn. It helps identify anomalies, formulate hypotheses and choose appropriate next analyses.

```
df.shape
df.head()
df.info()
df.describe(include="all")
df.isna().sum()
df.duplicated().sum()
```

Follow-up / counter-question: What is the difference between EDA and data cleaning?

Suggested answer: Cleaning fixes or standardizes data quality problems; EDA investigates structure and patterns. They are iterative.

Follow-up / counter-question: Is EDA only about charts?

Suggested answer: No. It includes summaries, counts, quality checks, grouping, comparisons and visualizations.

25. What steps do you follow for EDA on a new dataset?

Answer: Clarify the business question and row-level grain; load and inspect; check data types, missing values, duplicates, key integrity and business rules; clean carefully; examine distributions and relationships; validate findings; then communicate insights and limitations.

Follow-up / counter-question: Would you remove outliers before EDA?

Suggested answer: Not automatically. Investigate whether they are errors, valid extremes or a separate population.

Follow-up / counter-question: How do you validate cleaning?

Suggested answer: Compare before/after quality metrics, row counts, key uniqueness, business rules and important totals.

26. How do you identify outliers?

Answer: Common methods include IQR fences, z-scores and visual inspection. They flag unusual values, but an outlier is not necessarily an error.

```
q1 = df["sales"].quantile(0.25)
q3 = df["sales"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
outliers = df[(df["sales"] < lower) | (df["sales"] > upper)]
```

Follow-up / counter-question: What are IQR fences?

Suggested answer: IQR = Q3 − Q1. Conventional fences are Q1 − 1.5×IQR and Q3 + 1.5×IQR.

Follow-up / counter-question: How does IQR differ from a z-score?

Suggested answer: IQR uses quartiles and is more robust to extremes; z-scores use the mean and standard deviation.

Follow-up / counter-question: Should every outlier be deleted?

Suggested answer: No. Investigate context, source, units and business rules; retain, correct, flag, transform or exclude based on evidence.

27. What is the difference between univariate, bivariate and multivariate analysis?

Answer: Univariate analysis examines one variable; bivariate analysis examines two variables; multivariate analysis considers several variables together. Examples include order-value distribution, delivery time versus rating, and sales by region, category and month.

Follow-up / counter-question: Is correlation always bivariate?

Suggested answer: A correlation between two variables is bivariate; a correlation matrix explores many pairwise relationships.

Follow-up / counter-question: Does correlation prove causation?

Suggested answer: No. Confounding, reverse causality or coincidence may explain an association.

28. What is Seaborn and why is it used for EDA?

Answer: Seaborn is a statistical visualization library built on Matplotlib. It offers concise functions for distributions, relationships, categorical comparisons and heatmaps, with useful defaults.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(data=df, x="sales", kde=True)
plt.show()

sns.boxplot(data=df, x="category", y="sales")
plt.show()
```

Follow-up / counter-question: What is the difference between `countplot()` and `barplot()`?

Suggested answer: `countplot()` counts category observations; `barplot()` generally estimates a statistic such as mean sales.

Follow-up / counter-question: When can a mean bar chart mislead?

Suggested answer: Group sizes and outliers affect means. Consider displaying counts, uncertainty or full distributions.

29. What is the difference between a histogram, box plot and scatter plot?

Answer: A histogram shows a numerical distribution using bins; a box plot summarizes median, quartiles and whiskers while flagging potential outliers; a scatter plot displays paired numerical values to investigate relationships, clusters and unusual points.

Follow-up / counter-question: Which plot would you use for sales versus advertising spend?

Suggested answer: A scatter plot because both variables are numerical.

Follow-up / counter-question: Can a histogram show the relationship between two variables?

Suggested answer: A standard histogram shows one numerical variable. A 2D histogram or scatter plot is more appropriate for two variables.

30. What is Matplotlib and how is it different from Seaborn?

Answer: Matplotlib is a foundational plotting library with detailed control over chart elements. Seaborn builds on Matplotlib and makes common statistical plots easier to create. They are often used together.

```
plt.figure(figsize=(8, 4))
plt.plot(df["month"], df["sales"], marker="o")
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()
```

Follow-up / counter-question: Why use `plt.show()`?

Suggested answer: It explicitly requests plot display, especially useful in scripts; notebooks may display plots automatically.

Follow-up / counter-question: Which library should you use?

Suggested answer: Use Seaborn for concise statistical exploration and Matplotlib for detailed customization; choose based on the chart and task.

4. Core Python questions

31. What are Python's main built-in data types?

Answer: Common types include int, float, str, bool, list, tuple, set, dict and NoneType.

```
x = 10
name = 'Asha'
items = [1, 2, 3]
record = {'city': 'Surat'}
```

Follow-up / counter-question: Which are mutable?

Suggested answer: Lists, dictionaries and sets are mutable; tuples are immutable as containers, though they can contain mutable objects.

32. What is the difference between list, tuple, set and dictionary?

Answer: Lists are ordered and mutable; tuples are ordered and immutable; sets contain unique elements; dictionaries map keys to values.

```
items = [1, 2, 2]
point = (1, 2)
unique = set(items)
record = {'city': 'Surat'}
```

Follow-up / counter-question: Why use a set in analysis?

Suggested answer: For uniqueness checks and membership testing, but not when you need duplicate counts or frequency distributions.

33. What is the difference between == and is?

Answer: `==` compares values for equality; `is` checks whether two references point to the same object.

```
a = [1, 2]
b = [1, 2]
print(a == b)  # True
print(a is b)  # False
```

Follow-up / counter-question: How should you check for None?

Suggested answer: Use `value is None`.

34. What is a list comprehension?

Answer: A concise way to construct a list from an iterable, optionally applying a condition.

```
numbers = [1, 2, 3, 4, 5]
squares = [n**2 for n in numbers if n % 2 == 0]
```

Follow-up / counter-question: Should you always use one?

Suggested answer: No. A normal loop may be clearer for complicated logic; a generator can save memory if results are consumed one at a time.

35. What are functions, and why are they useful?

Answer: Functions package reusable logic, making analysis easier to test, maintain and explain.

```
def profit(revenue, cost):
    return revenue - cost

print(profit(1000, 600))
```

Follow-up / counter-question: Difference between return and print?

Suggested answer: `return` passes a value back to the caller; `print()` displays a value.

36. What is exception handling?

Answer: It lets a program respond to expected errors using try/except, with finally often used for cleanup.

```
try:
    number = int('123')
```

```
except ValueError:
    print('Invalid number')
```

Follow-up / counter-question: Should you use a bare except for every error?

Suggested answer: No. Catch expected exceptions so programming mistakes are not hidden.

37. What is a shallow copy versus a deep copy?

Answer: A shallow copy creates a new outer container but can share nested objects. A deep copy recursively copies nested objects.

```
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)
```

Follow-up / counter-question: Why does it matter?

Suggested answer: Changing a nested object may affect both the original and shallow copy, but normally not the deep copy.

38. What is a lambda function?

Answer: A small anonymous function containing a single expression. It is useful for short transformations, but complex logic is usually clearer as a named function.

```
square = lambda x: x ** 2
print(square(5))
```

Follow-up / counter-question: Can a lambda contain multiple statements?

Suggested answer: No; a lambda body is a single expression.

39. What is the difference between map, filter and Pandas apply?

Answer: `map()` transforms iterable elements; `filter()` keeps elements satisfying a condition; Pandas `apply()` applies a function across a Series or DataFrame axis.

```
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x*x, nums))
evens = list(filter(lambda x: x % 2 == 0, nums))
```

Follow-up / counter-question: Is apply always fastest?

Suggested answer: No. Vectorized Pandas/NumPy operations are often faster and clearer for common calculations.

5. Data cleaning and practical Pandas questions

40. What is vectorization?

Answer: Vectorization applies an operation to a whole array or Series rather than manually looping through every element. It often improves performance and readability.

```
df['profit'] = df['revenue'] - df['cost']
```

Follow-up / counter-question: Is vectorization always possible?

Suggested answer: Not for every custom operation; use an appropriate method and profile if performance matters.

41. Difference between map(), apply() and replace() in Pandas?

Answer: `Series.map()` maps each value using a dictionary, Series or function; `apply()` applies a function along a Series/DataFrame axis; `replace()` substitutes specified values or patterns.

```
df['status'] = df['status'].replace({'deliv': 'delivered', 'cancel': 'canceled'})
```

Follow-up / counter-question: Why not replace every unexpected category automatically?

Suggested answer: Unexpected values may be valid new categories, aliases or errors. Inspect them before standardizing.

42. How do you change a column's data type?

Answer: Use `astype()` for compatible conversions, `pd.to_numeric()` for numeric parsing, and `pd.to_datetime()` for date parsing. Inspect failures when coercing invalid values.

```
df['sales'] = pd.to_numeric(df['sales'], errors='coerce')
df['date'] = pd.to_datetime(df['date'], errors='coerce')
```

Follow-up / counter-question: Why can conversion fail?

Suggested answer: Values may include text, symbols, invalid dates, mixed formats or missing values.

43. Difference between pivot() and pivot_table()?

Answer: `pivot()` reshapes data but requires each index/column combination to be unique. `pivot_table()` can aggregate repeated combinations.

```
pd.pivot_table(df, index='city', columns='category', values='sales', aggfunc='sum', fill_value=0)
```

Follow-up / counter-question: Why might pivot raise an error?

Suggested answer: Repeated index-column combinations make the result ambiguous; aggregate first or use pivot_table.

44. What are melt() and wide-to-long reshaping used for?

Answer: `melt()` turns wide data with repeated measure columns into long data with a variable column and a value column.

```
long_df = pd.melt(df, id_vars=['product'], value_vars=['jan_sales', 'feb_sales'], var_name='month', value_name='sales')
```

Follow-up / counter-question: Why reshape wide data to long?

Suggested answer: Long format is often easier for grouping, plotting, and working with repeated measures.

45. What is a MultiIndex?

Answer: A hierarchical index with multiple levels, useful for grouped summaries, pivot results and hierarchical selection.

```
df = df.reset_index()
```

Follow-up / counter-question: How do you turn index levels into columns?

Suggested answer: Use `reset_index()`; use `drop=True` if you do not need the old index.

46. Difference between transform() and agg()?

Answer: `agg()` usually reduces groups to summaries; `transform()` returns results aligned to the original rows.

```
df['category_avg'] = df.groupby('category')['sales'].transform('mean')
above = df[df['sales'] > df['category_avg']]
```

Follow-up / counter-question: How do you compare each row with its group average?

Suggested answer: Create a group mean using transform, then compare the row value to that result.

47. How do you find inconsistent text values?

Answer: Inspect unique values and frequencies, standardize whitespace/case where appropriate, and use a reviewed mapping for known misspellings or aliases.

```
df['city'] = df['city'].astype('string').str.strip().str.title()
```

Follow-up / counter-question: Will title-casing fix all city-name issues?

Suggested answer: No. It will not identify all misspellings or legitimate capitalization exceptions.

48. How do you handle inconsistent date formats?

Answer: Inspect source formats and parse dates with a format or options matching the actual data. Coercing failures can help identify invalid entries, but ambiguous dates require care.

```
df['date'] = pd.to_datetime(df['date'], errors='coerce', dayfirst=True)
```

Follow-up / counter-question: Why is 03/04/2026 risky?

Suggested answer: It can mean March 4 or April 3 depending on the source convention.

49. How do you detect invalid values using business rules?

Answer: Define rules from the domain, such as non-negative quantity, valid review-score range, delivery after order date, and valid foreign keys. Flag violations for investigation.

```
invalid_dates = df[df['delivery_date'] < df['order_date']]
```

Follow-up / counter-question: Should invalid records be deleted immediately?

Suggested answer: No. Establish whether it is a true error or valid exception, document the decision, and check impact.

50. How do you detect issues after merging datasets?

Answer: Compare row counts, key uniqueness, expected relationship, unmatched keys, new nulls, and important totals. Use merge validation and an indicator where useful.

```
merged = orders.merge(customers, on='customer_id', how='left', validate='many_to_one', indicator=True)
print(merged['_merge'].value_counts())
```

Follow-up / counter-question: Why use validate='many_to_one'?

Suggested answer: It ensures the right-side key is unique for a many-to-one relationship, helping prevent accidental row multiplication.

6. Statistics and analytical reasoning

51. Difference between mean, median and mode?

Answer: Mean is the arithmetic average; median is the middle sorted value; mode is the most frequent value or values. Choose based on distribution and business meaning.

Follow-up / counter-question: Which is most robust to extreme values?

Suggested answer: The median is generally more robust than the mean.

52. What is standard deviation?

Answer: It measures spread around the mean in the original measurement units. Larger standard deviation indicates greater dispersion, though context and scale matter.

Follow-up / counter-question: Can standard deviations be compared across differently scaled variables?

Suggested answer: Not directly in all cases; consider scale and context, or a normalized measure when appropriate.

53. What is variance?

Answer: Variance is the average squared deviation from the mean. Population variance divides by N; sample variance commonly divides by n−1.

Follow-up / counter-question: Why square the deviations?

Suggested answer: Squaring prevents positive and negative deviations from canceling and gives more weight to larger deviations.

54. What is correlation?

Answer: Correlation measures direction and strength of association. Pearson correlation captures linear association and ranges from −1 to +1.

Follow-up / counter-question: Does zero Pearson correlation mean no relationship?

Suggested answer: No. A nonlinear relationship can have Pearson correlation near zero.

55. Difference between correlation and covariance?

Answer: Covariance indicates whether variables tend to move together but depends on units. Correlation standardizes the relationship to a scale from −1 to +1 for Pearson correlation.

Follow-up / counter-question: Which is easier to compare across datasets?

Suggested answer: Correlation is generally easier because it is standardized.

56. What is a normal distribution?

Answer: A symmetric, bell-shaped continuous distribution described by its mean and standard deviation.

Follow-up / counter-question: Are real datasets always normally distributed?

Suggested answer: No. Sales, income and delivery times may be skewed or multimodal; inspect the data.

57. What is hypothesis testing?

Answer: A statistical procedure uses sample data to evaluate evidence against a null hypothesis, considering assumptions, test statistic, p-value and significance threshold.

Follow-up / counter-question: Does a small p-value prove the alternative is true?

Suggested answer: No. It measures incompatibility with the null model under its assumptions; it does not prove causation or practical importance.

58. Population versus sample?

Answer: A population is the complete group of interest; a sample is a subset used to estimate or understand that population.

Follow-up / counter-question: Why does sampling method matter?

Suggested answer: A biased or unrepresentative sample can lead to misleading generalizations.

59. What is sampling bias?

Answer: Sampling bias occurs when the data collection process systematically over- or under-represents parts of the target population.

Follow-up / counter-question: Give an example.

Suggested answer: Surveying only highly active customers may misrepresent satisfaction among all customers.

60. Statistical versus practical significance?

Answer: Statistical significance concerns evidence against a null hypothesis; practical significance concerns whether the effect matters for a real-world decision.

Follow-up / counter-question: Can a tiny effect be statistically significant?

Suggested answer: Yes, especially with a very large sample, even if the effect is commercially unimportant.

7. Python coding questions for analysts

61. How do you find duplicate records?

Answer: Use `duplicated()` to flag duplicates. Specify a subset based on the intended key if full-row equality is not the business rule.

```
df[df.duplicated(keep=False)]
df[df.duplicated(subset=['customer_id'], keep=False)]
```

Follow-up / counter-question: How do you remove duplicates safely?

Suggested answer: Confirm the table grain and retention rule first; never assume repeated IDs are automatically invalid.

62. How do you find the top five highest-selling products?

Answer: Group by product, sum sales, sort descending and take the first five. Clarify whether the requirement includes ties at the cutoff.

```
top_products = (df.groupby('product_name')['sales'].sum().sort_values(ascending=False).head(5))
```

63. How do you calculate monthly revenue?

Answer: Convert the order date to datetime, group by month and sum revenue. Ensure date parsing and time zone conventions are correct.

```
df['order_date'] = pd.to_datetime(df['order_date'])
monthly = df.groupby(df['order_date'].dt.to_period('M'))['revenue'].sum()
```

Follow-up / counter-question: What if some months have no records?

Suggested answer: Create a complete monthly index and decide whether absent months mean zero activity or unknown data.

64. How do you calculate month-over-month growth?

Answer: Sort monthly revenue chronologically and use `pct_change() * 100`. The first month has no prior comparison; a zero prior value makes percentage growth undefined.

```
monthly = monthly.sort_index()
growth_pct = monthly.pct_change() * 100
```

65. How do you find the highest salary in each department?

Answer: Group by department and calculate maximum salary. To return employee names, join the maxima back to employee rows and retain all matching ties.

```
max_salary = df.groupby('department')['salary'].max().rename('max_salary')
result = df.merge(max_salary, on='department')
result = result[result['salary'] == result['max_salary']]
```

66. How do you calculate a running total?

Answer: Sort rows by the relevant order before using `cumsum()`. For separate customers, use groupby followed by cumsum.

```
df = df.sort_values('order_date')
df['running_total'] = df['sales'].cumsum()
df['customer_running_total'] = df.groupby('customer_id')['sales'].cumsum()
```

67. How do you calculate a rolling three-month average?

Answer: First create one observation per month, sort it, and then use a rolling window of three observations. Check how missing months should be handled.

```
monthly_avg = monthly_revenue.rolling(3).mean()
```

Follow-up / counter-question: Is a three-row window always three months?

Suggested answer: No. Rows must represent consecutive months for that interpretation.

68. How do you find customers who have never placed an order?

Answer: Compare customer IDs with order customer IDs using `isin()` and negate the result, or use a left merge indicator.

```
customers_without_orders = customers[~customers['customer_id'].isin(orders['customer_id'])]
```

69. How do you calculate each category's percentage of revenue?

Answer: Sum revenue by category and divide each category's total by the overall total, multiplying by 100. Define how returns and negative values should be treated.

```
category_rev = df.groupby('category')['revenue'].sum()
category_pct = category_rev / category_rev.sum() * 100
```

70. How do you summarize missing values?

Answer: Calculate missing counts and percentages by column, then investigate whether missingness clusters by status, time, segment or process.

```
missing = pd.DataFrame({'count': df.isna().sum(), 'pct': df.isna().mean()*100})
missing = missing.sort_values('pct', ascending=False)
```

8. Scenario-based interview questions

71. A dataset has 20% missing values. What would you do?

Answer: Inspect missingness by column and segment, understand the business process, quantify impact, and decide whether to retain, impute, flag or remove records. Document the decision and validate the resulting data.

Follow-up / counter-question: Would you immediately use dropna()?

Suggested answer: No. It may discard a large or systematically different part of the dataset.

72. Your dataset contains duplicate customer IDs. Is that an error?

Answer: Not necessarily. Determine the table grain. A customer master may require one row per customer; an orders table may legitimately have many rows per customer.

73. Row count doubles after a merge. What do you check?

Answer: Inspect duplicate join keys, expected relationship, join type, unmatched records, and pre/post row counts. Test whether the merge accidentally became many-to-many.

74. Average sales increases after cleaning. Why?

Answer: Possible causes include removing low-value rows, imputing values, correcting negative entries, deleting duplicates or changing filters. Identify which transformation caused the change and whether it was justified.

75. A manager asks you to remove all outliers. What do you say?

Answer: Explain that outliers may be errors or genuine high-value cases. Investigate them, quantify the effect of alternative treatments, and recommend a method aligned with the business objective.

76. Python results differ from SQL results. How do you investigate?

Answer: Compare filters, join keys and join types, duplicate handling, NULL/NaN behavior, date/time-zone interpretation, numeric types, aggregation order and metric definitions.

77. How would you investigate declining monthly sales?

Answer: Validate data completeness, compare revenue/order count/average order value, segment by product and region, investigate returns, discounts, cancellations and seasonality, then identify the largest contributors.

Follow-up / counter-question: Does declining revenue always mean fewer orders?

Suggested answer: No. Average order value, product mix, discounts, returns and other factors can change revenue.

78. How do you present EDA findings to a non-technical manager?

Answer: Lead with the business question, communicate the key finding and its magnitude, use simple charts, explain limitations, and recommend actionable next steps.

9. Rapid revision: short questions

What does `axis=0` mean in a 2D NumPy reduction?

Answer: Reduces along rows, commonly producing one result per column.

What does `axis=1` mean?

Answer: Reduces across columns, commonly producing one result per row.

What is NaN?

Answer: A floating-point representation commonly used for missing or invalid numeric values.

What is NaT?

Answer: A missing-value representation for date/time values in Pandas.

What is `value_counts()`?

Answer: Counts occurrences of distinct values in a Series.

What is `nunique()`?

Answer: Counts unique values, excluding missing values by default.

What does `reset_index()` do?

Answer: Resets the index; optionally retains the old index as a column.

What does `inplace=True` mean?

Answer: Requests modification of the object rather than returning a separate modified result, where supported.

What is `cut()`?

Answer: Bins numerical values into specified intervals.

What is `qcut()`?

Answer: Creates quantile-based bins, subject to ties and duplicate boundaries.

What is a categorical dtype?

Answer: A dtype for variables with a limited set of categories.

What is a correlation heatmap?

Answer: A color-coded matrix, often used to visualize pairwise correlations.

What is `random_state`?

Answer: A seed/state setting that helps make supported randomized operations reproducible.

What is `resample()`?

Answer: Groups time-indexed data into time frequencies for aggregation or related operations.

What is `shift()`?

Answer: Moves values along an axis; often used to compare current and prior observations.

What is `diff()`?

Answer: Calculates differences between values, often consecutive observations.

What is `rank()`?

Answer: Assigns ranks to values using a selected ranking method.

What is `explode()`?

Answer: Expands list-like elements into multiple rows.

What is `clip()`?

Answer: Limits values to a lower and/or upper bound.

What is `corr()`?

Answer: Calculates pairwise correlations for supported columns or Series.

What is a residual?

Answer: The difference between an observed value and a fitted or predicted value.

What is data leakage?

Answer: When information unavailable at prediction time improperly enters model training or evaluation.

What is reproducibility?

Answer: The ability to repeat an analysis with consistent results using the same data, code and relevant settings.

Why use Git?

Answer: To track code changes, collaborate, review history and restore earlier versions.

What is an API?

Answer: A defined interface through which software systems exchange data or functionality.

What is a virtual environment?

Answer: An isolated Python environment for managing project-specific dependencies.

What is a Jupyter Notebook?

Answer: An interactive document combining executable code, output, visualizations and explanatory text.

What is a CSV file?

Answer: A text format that stores tabular data using delimiters, commonly commas.

What is an inner join?

Answer: Returns records with matching keys in both tables.

What is a left join?

Answer: Keeps all left-side rows and adds matching right-side data.

10. Final preparation checklist

☐ Explain the difference between a list and a NumPy array, including vectorization.

☐ Create arrays and explain shape, ndim, size, dtype and broadcasting.

☐ Calculate mean, median, standard deviation and variance; explain ddof.

☐ Load a CSV and inspect shape, types, missing values, duplicates and summary statistics.

☐ Filter rows using multiple conditions, and distinguish loc from iloc.

☐ Group data and calculate sums, means, counts and percentages.

☐ Merge datasets while checking key uniqueness, row counts and unmatched records.

☐ Handle missing values and outliers based on evidence rather than automatic rules.

☐ Choose an appropriate chart for a distribution, category comparison or relationship.

☐ Explain correlation versus causation and statistical versus practical significance.

☐ Solve monthly revenue, growth, running-total and top-category coding tasks.

☐ Describe one project end to end: objective, data, cleaning, analysis, validation, findings and limitations.

☐ Practise explaining code line by line without relying on memorized phrases.

☐ Use GitHub or another version-control tool to maintain a clear project history.

A strong answer pattern

1) Define the concept. 2) Explain when and why you would use it. 3) Give a short example. 4) Mention one limitation or validation check.

Suggested study method: revise a small set of questions each day, write code from memory, test it on a sample dataset, and revisit questions you could not explain clearly after a few days.